MaxEntIRL HW2

ayalla@umass.edu

Question 1: Selecting the Learning Rate

To determine an appropriate static learning rate for the gradient descent optimization, I conducted an empirical analysis by tracking the L_2 norm of the gradient over 25, 50, 100 epochs for various step sizes. A theoretically sound learning rate should allow the algorithm to smoothly converge to a minimum (where $||\nabla L(\theta)||_2 \approx 0$) without oscillating wildly or terminating prematurely due to numerical overflow.

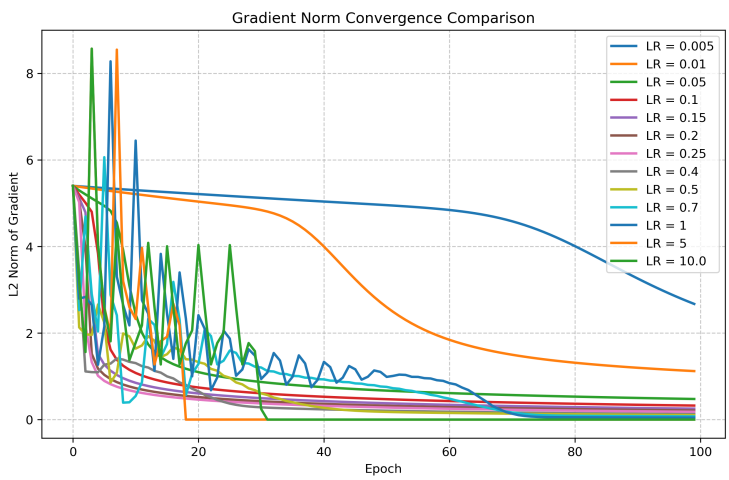

Figure 1: Gradient norm convergence over 100 epochs across varying learning rates.

As shown in Figure 3, the optimization dynamics exhibit three distinct phases depending on the step size:

- **Undershooting (e.g., LR = 0.005, 0.01):** The gradient norm decays too slowly. While stable, the algorithm fails to reach the true reward scale within the 100-epoch time limit, resulting in a low-confidence, high-entropy policy that does not distinctly separate good trajectories from bad ones.

- **Overshooting and Instability (e.g., LR $\geq$ 0.7):** These learning rates cause severe oscillations in the early epochs as the optimizer ricochets across the loss landscape. Furthermore, extremely high rates (e.g., LR = 5, 10) trigger catastrophic numerical overflow. The massive weight updates cause the exponentiated reward calculations ($e^{\theta^\top f}$) in the partition function to evaluate to infinity, instantly flattening the gradient to zero.

- **Stable Convergence (e.g., LR = 0.1 to 0.25):** This range represents the ideal step size. The gradient norm exhibits a smooth, monotonic, and exponential decay after a brief initial calibration.

Based on this analysis, I selected a learning rate of **0.2** for the main experiment, as it provided the fastest stable convergence without risking numerical instability.

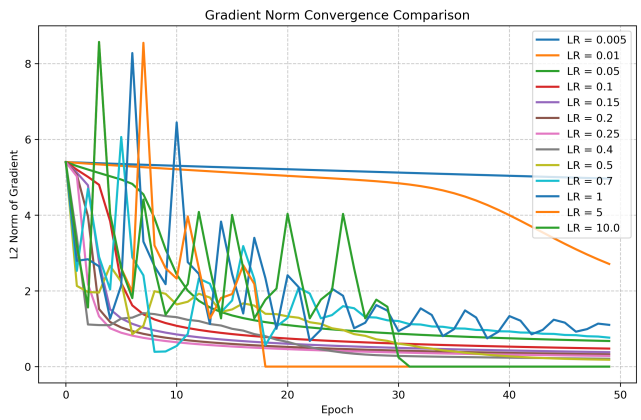

Figure 2: Gradient norm convergence over 50 epochs across varying learning rates.

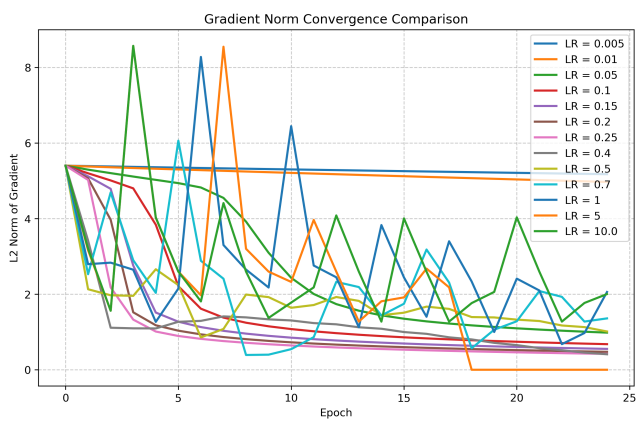

Figure 3: Gradient norm convergence over 25 epochs across varying learning rates.

Question 2: Reward Function Shape and State 12

Running the MaxEnt IRL algorithm for 100 epochs with a horizon of 15 (using a learning rate of 0.2) yields a distinct topological reward landscape.

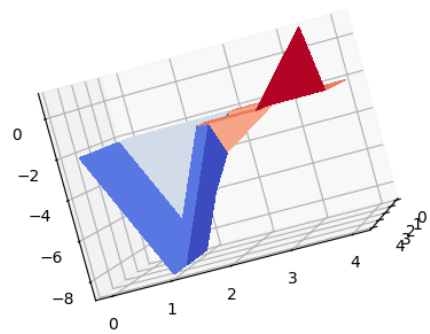

Figure 4: 3D surface plot of the learned reward function landscape.

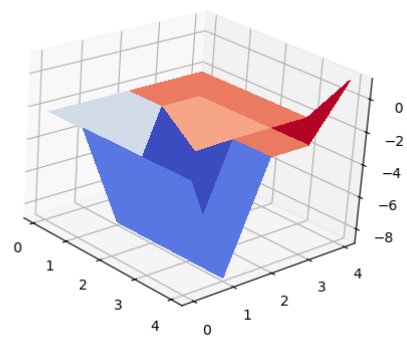

Figure 5: 3D surface plot of the learned reward function landscape.

Qualitative Shape and Goal: As illustrated in Figure 5, the learned reward function features a prominent positive peak at coordinate (4, 4) corresponding to the terminal goal (state 24), a severe negative trench across the second row ($y = 1$, states 6-9), and a slight, uniform negative penalty across

the default terrain. Qualitatively, the algorithm has learned to heavily penalize the "obstacle" row, strongly reward the goal state, and apply a step-cost to the normal ground. It deduced this because the expert trajectories consistently terminate at the goal while taking intentionally longer, circuitous routes along the edges of the grid specifically to avoid the second row.

The Treatment of State 12: In the center of the grid, state 12 is assigned a significant negative reward (creating a "sinkhole" in the landscape). Mathematically, this is a sensible deduction based *strictly* on the provided demonstrations. State 12 is geographically closer to the starting states than the outer edges. Because an optimal agent minimizing step-costs should have walked through state 12, the fact that the expert deliberately chose a longer path looping around the center serves as strong mathematical evidence to the algorithm that state 12 must contain a hazard.

However, with respect to the ground truth, this deduction might be entirely incorrect. This highlights the vulnerability of Inverse Reinforcement Learning to unvisited states and spurious correlations. The algorithm assumes that an absence of evidence (the expert never visiting state 12) is evidence of a penalty. In reality, state 12 could possess a high positive reward, but the expert simply did not know it was there, or the expert may have had an irrational preference for walking near the walls. Because the algorithm must extrapolate the value of state 12 without any direct demonstration data inside it, it confidently hallucinated a hazard that may not exist in the true environment.

Question 3: Gradient of the Loss Function

We are given the log-likelihood loss function for a single expert demonstration trajectory τ under deterministic dynamics:

$$L(\theta) = \log P(\tau|\theta) = \log \left[\frac{e^{\theta^\top f_\tau}}{Z(\theta)} \right]$$

where $Z(\theta)$ is the partition function defined as the sum over all possible trajectories ζ:

$$Z(\theta) = \sum_{\zeta} e^{\theta^\top f_\zeta}$$

Step 1: Apply logarithm rules
Using the quotient rule for logarithms, $\log(a/b) = \log(a) - \log(b)$, we can separate the numerator and the denominator:

$$L(\theta) = \log \left(e^{\theta^\top f_\tau} \right) - \log Z(\theta)$$

Since the natural log and the exponential function are inverses, the first term simplifies to just the exponent:

$$L(\theta) = \theta^\top f_\tau - \log Z(\theta)$$

Step 2: Differentiate with respect to θ
We take the gradient of the simplified loss function:

$$\nabla_\theta L(\theta) = \nabla_\theta \left(\theta^\top f_\tau\right) - \nabla_\theta \log Z(\theta)$$

The derivative of the first term with respect to θ is simply the feature counts of the demonstrated trajectory:

$$\nabla_\theta \left(\theta^\top f_\tau\right) = f_\tau$$

Step 3: Differentiate the partition function (Chain Rule)
To differentiate the second term, we apply the chain rule for logarithms, $\nabla \log(x) = \frac{1}{x}\nabla x$:

$$\nabla_\theta \log Z(\theta) = \frac{1}{Z(\theta)} \nabla_\theta Z(\theta)$$

Now, we substitute the definition of $Z(\theta)$ and push the gradient operator inside the summation:

$$\nabla_\theta Z(\theta) = \nabla_\theta \sum_\zeta e^{\theta^\top f_\zeta} = \sum_\zeta \nabla_\theta \left(e^{\theta^\top f_\zeta}\right)$$

Applying the chain rule for exponentials, $e^u \to e^u \nabla u$:

$$\nabla_\theta Z(\theta) = \sum_\zeta e^{\theta^\top f_\zeta} f_\zeta$$

We substitute this result back into our derivative of the log partition function:

$$\nabla_\theta \log Z(\theta) = \frac{1}{Z(\theta)} \sum_\zeta e^{\theta^\top f_\zeta} f_\zeta = \sum_\zeta \left[\frac{e^{\theta^\top f_\zeta}}{Z(\theta)}\right] f_\zeta$$

Recognizing that the term in the brackets is exactly the definition of the probability of a trajectory $P(\zeta|\theta)$, we can simplify this to:

$$\nabla_\theta \log Z(\theta) = \sum_\zeta P(\zeta|\theta) f_\zeta$$

This term represents the expected feature counts of our model under the current policy.

Step 4: Construct the final gradient
Substitute both differentiated terms back into the equation from Step 2:

$$\nabla_\theta L(\theta) = f_\tau - \sum_\zeta P(\zeta|\theta) f_\zeta$$

To finalize the derivation and match Equation 6 from the paper, we generalize from a single trajectory τ to multiple expert demonstrations. We replace the single trajectory features f_τ with the empirical average of the expert's feature expectations, denoted as $\tilde{f}$:

$$\nabla_\theta L(\theta) = \tilde{f} - \sum_\zeta P(\zeta|\theta) f_\zeta$$

Question 4:

The core difference between the Maximum Entropy policy (Equation 5) and the action-based model (Equation 7) comes down to how they handle normalization. Equation 5 normalizes over entire paths (globally), while Equation 7 normalizes at each individual state (locally).

Equation 5: The MaxEnt Policy

In Equation 5, the probability of a trajectory depends only on its total accumulated reward:

$$P(\zeta|\theta) = \frac{1}{Z(\theta)} e^{\theta^\top f_\zeta}$$

Because it normalizes globally over all possible complete paths using $Z(\theta)$, any paths with the same total reward are treated equally. In the Figure 2 example, there are four possible paths to the goal that all yield the exact same reward (one top path, three bottom paths). Equation 5 sees four mathematically equivalent paths and simply splits the probability evenly, assigning exactly 25% to each.

Equation 7: The Action-Based Model

The action-based model, however, calculates probabilities step-by-step:

$$P(\zeta) = \prod_{s_t, a_t \in \zeta} P(a_t|s_t)$$

Because probabilities must sum to 1 at every single state, this model normalizes locally. At the very first state in Figure 2, the agent has two choices (up or down), so it assigns a 50% probability to the first step of the top route and 50% to the bottom route. As the agent moves along the bottom route, it hits more intersections. That initial 50% probability mass gets fractured and divided again and again at every branch. Consequently, the single unbranched top path holds onto its massive 50% probability, while the three equivalent bottom paths are unfairly diluted down to roughly 16.6% each.

Why the Weighting of Equation 7 is Problematic

The weighting in Equation 7 is problematic because it penalizes paths simply for having more branching decisions. In Inverse Reinforcement Learning, we want to recover the expert's underlying reward function. An expert's preferred route shouldn't be assigned a lower probability by our model just because the environment happens to have more intersecting roads along that specific path.

If we use Equation 7, the learned probabilities become an artifact of how the state space is structured (i.e., how many actions are available at each state) rather than reflecting the actual reward values the expert is optimizing. Equation 5 avoids this completely by ignoring the branching structure of the MDP and focusing solely on the total reward of the trajectory.

Question 5:

In the Maximum Entropy IRL framework, Z_{s_i} and $Z_{a_{i,j}}$ function as local partition functions that aggregate the rewards of all possible future trajectories over

a given horizon.

- Z_{s_i} **(State Partition Function):** This value represents the total exponentiated reward-weight of all paths starting from state s_i. Intuitively, it serves as a "soft" version of a Value Function. While a standard Value Function $V(s)$ tracks the maximum possible reward from a state, Z_{s_i} tracks the sum of all possible path rewards, consistent with the principle of Maximum Entropy. It is a measure of the total desirability of being in state s_i. If Z_{s_i} is high, it means there are many high-reward paths available to the agent from that state.

- $Z_{a_{i,j}}$ **(Action Partition Function):** This represents the total weight of all paths that begin with a specific action a from state s_i. It captures the desirability of an action by accounting for the immediate reward and the weighted sum of all future states reachable from that action. It evaluates the quality of a specific decision. It answers the question: "Given that I just committed to action a in state s_i, what is the sum of all the rewards I can expect to collect from my next state s_j until the end of the horizon?"

Over a particular horizon, these values allow us to compute the stochastic policy $P(a|s) = \frac{Z_{a_{i,j}}}{Z_{s_i}}$. This ensures that the probability of an action is proportional to the total reward-weight of the paths it enables, allowing the model to capture the expert's preference for high-reward trajectories while maintaining the highest possible entropy for paths of equal utility.

Question 6:

In Step 2 of Algorithm 1, the value of 1 is added to Z_{s_i} for terminal states to account for the "return" of trajectories that have reached their destination.

Mathematically, Z represents the sum of the exponentiated rewards of all possible future paths. When an agent is at the terminal state s_{terminal}, the trajectory is complete, and the remaining reward to be collected is 0. Since the partition function is calculated in the exponentiated reward space, the contribution of a completed trajectory with zero remaining reward is $e^0 = 1$.

By adding this constant to the terminal state during each iteration of the backward pass, the algorithm ensures that Z_{s_i} correctly accumulates the weights of trajectories of all possible lengths that terminate within the given horizon. Without this +1 term, the partition function would fail to recognize the terminal state as a valid endpoint, and the probability mass for paths reaching the goal would not be properly normalized.